

# SMARTCAMPUS PARKING PFLICHTENHEFT

Projekt: SmartCampus Parking

Dokumentstatus: Beispiel / Testprojekt für SOPHub

Version: 1.0

Stand: 23.09.2026

Zweck: Realistisches Beispieldokument zur Darstellung eines abgeschlossenen Softwareprojekts.

## 1. Zweck und Ziel des Dokuments

Dieses Pflichtenheft beschreibt die technische Umsetzung des Projekts SmartCampus Parking. Es basiert auf den Anforderungen aus dem Lastenheft und definiert die konkrete Realisierung des Softwaresystems.

## 2. Systemübersicht

SmartCampus Parking ist eine webbasierte Anwendung zur Verwaltung von Hochschulparkplätzen. Das System besteht aus drei Hauptkomponenten: Frontend:

- React Anwendung
- Benutzeroberfläche für Studierende und Administratoren
- Darstellung von Parkplätzen und Reservierungen Backend:
- Spring Boot REST API
- Geschäftslogik
- Verarbeitung von Benutzeranfragen Datenbank:
- PostgreSQL
- Speicherung von Benutzern, Fahrzeugen, Parkplätzen und Reservierungen

### 3. Systemarchitektur

Die Anwendung verwendet eine klassische Client-Server-Architektur. Ablauf: Benutzer

↓ React Frontend

↓ REST API

↓ Spring Boot Backend

↓ PostgreSQL Datenbank Das Backend übernimmt Validierung, Geschäftsregeln und Datenzugriffe.

### 4. Technische Umsetzung

Frontend:

- React
- React Router
- Bootstrap für Benutzeroberfläche Backend:
- Spring Boot
- REST Controller
- Service Layer
- Repository Layer Datenbank:
- PostgreSQL
- Relationale Datenstruktur

### 5. Benutzerverwaltung

Das System unterstützt verschiedene Rollen: Student:

- Parkplatz suchen
- Fahrzeug verwalten
- Reservierungen erstellen Mitarbeiter:
- Nutzung von Parkplätzen Administrator:
- Verwaltung von Parkflächen
- Verwaltung von Benutzern

### 6. Umsetzung der Parkplatzverwaltung

Administratoren können Parkplätze verwalten. Funktionen:

- Parkplatz erstellen
- Parkplatz ändern
- Parkplatz deaktivieren
- Status aktualisieren Ein Parkplatz besitzt Informationen wie:
- Nummer
- Standort
- Typ
- aktueller Status

## 7. Umsetzung des Reservierungssystems

Ablauf einer Reservierung: 1. Benutzer wählt Zeitraum. 2. System sucht verfügbare Parkplätze. 3. Benutzer bestätigt Auswahl. 4. Backend prüft Verfügbarkeit. 5. Reservierung wird gespeichert.

Mögliche Status:

- ANGEFRAGT
- BESTÄTIGT
- STORNIERT

## 8. REST API Beschreibung

Beispiele für Schnittstellen: GET /api/parking-spaces Beschreibung: Liefert verfügbare Parkplätze. POST /api/reservations Beschreibung: Erstellt eine neue Reservierung. GET /api/reservations/{id} Beschreibung: Zeigt Details einer Reservierung. DELETE /api/reservations/{id} Beschreibung: Storniert eine Reservierung.

## 9. Sicherheitskonzept

Das System berücksichtigt folgende Sicherheitsmaßnahmen:

- Authentifizierung von Benutzern
- Rollenbasierte Berechtigungen
- Schutz personenbezogener Daten
- Keine Speicherung von Passwörtern im Klartext
- Prüfung von Benutzerrechten vor Aktionen

## 10. Datenverarbeitung

Gespeicherte Daten: Benutzer:

Name, E-Mail, Rolle Fahrzeuge:

Kennzeichen, Fahrzeugtyp Parkplätze:

Standort, Nummer, Status Reservierungen:

Zeitraum, Benutzer, Parkplatz, Status

## **11. Testkonzept**

Folgende Tests werden durchgeführt: Test 1: Benutzer meldet sich an. Erwartung: Benutzer erhält Zugriff. Test 2: Benutzer erstellt Reservierung. Erwartung: Reservierung wird gespeichert. Test 3: Administrator erstellt Parkplatz. Erwartung: Parkplatz erscheint in Übersicht.

## **12. Deployment und Betrieb**

Die Anwendung wird als Webanwendung betrieben. Benötigte Komponenten:

- Webserver
- Backend Server
- PostgreSQL Datenbank Regelmäßige Backups der Datenbank werden empfohlen.

## **13. Zusammenfassung**

SmartCampus Parking wird als moderne Webanwendung umgesetzt. Die Architektur ermöglicht Erweiterungen und eine einfache Wartung. Das System unterstützt Benutzer bei der Parkplatzsuche und Verwaltung.